



NivoTalk

Complete App Synopsis

Every feature and function — what it does, how it was built, and how it performs

Product	Messenger with a built-in multiplayer photobooth
Platforms	Web (mobile-first, desktop) and Android app
Live site	https://nivotalk.onrender.com
Repository	github.com/simmi2k25/NivoTalk · main @ 0600084
Document date	6 October 2026

Contents

1 Overview	13 QR ID
2 At a glance	14 Design system, theme & motion
3 Architecture	15 Server wake-up handling
4 Technology stack	16 Android app
5 Accounts & sign-in	17 Data model
6 Friends & presence	18 API & real-time reference
7 Chat	19 Security & privacy
8 Stickers	20 Performance
9 The live photobooth	21 Build, run & deploy
10 Strip editor & Memories	22 How NivoTalk was created
11 Profiles, photos & bio	23 Known limitations & roadmap
12 Profile songs	

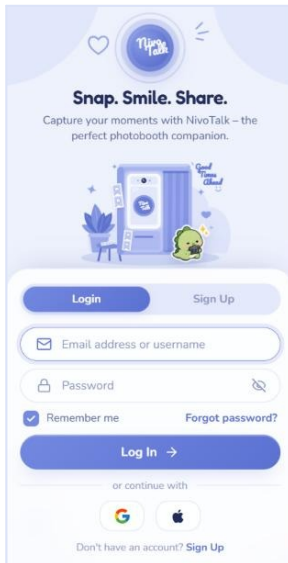
SECTION 1

Overview

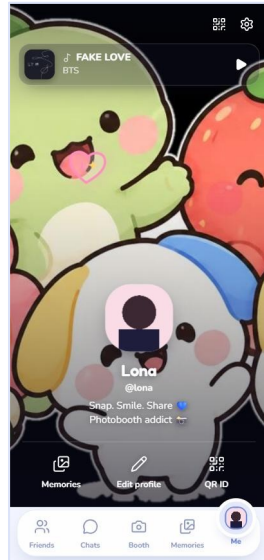
NivoTalk is a cute, character-filled messenger with a photobooth built in. Friends chat with stickers, then hop into a booth room together — side by side on live video, from anywhere — and take a shared photo strip that everyone can decorate, keep and share.

- **Who it's for:** friends and small groups who want something warmer and more playful than a plain chat app.
- **Core idea:** chat → invite to a booth → a synced countdown fires every camera at the same moment → one strip with everyone in it.
- **Personality:** five sticker characters (Berri, Chatty, Dino, Pepo, Pomi), a periwinkle look taken from the NivoTalk logo, and gentle animation everywhere.
- **Where it runs:** any modern browser (phone or desktop) and as an installable Android app.

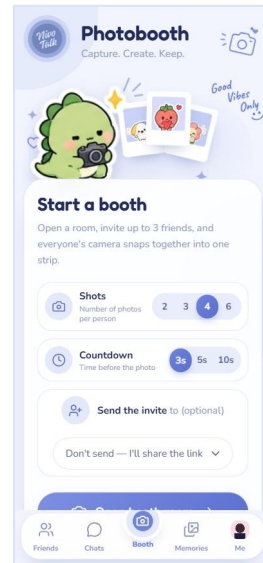
Main screens



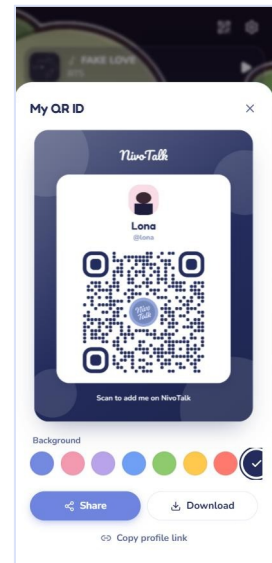
Login / sign-up



Profile with song



Photobooth home



QR ID

SECTION 2

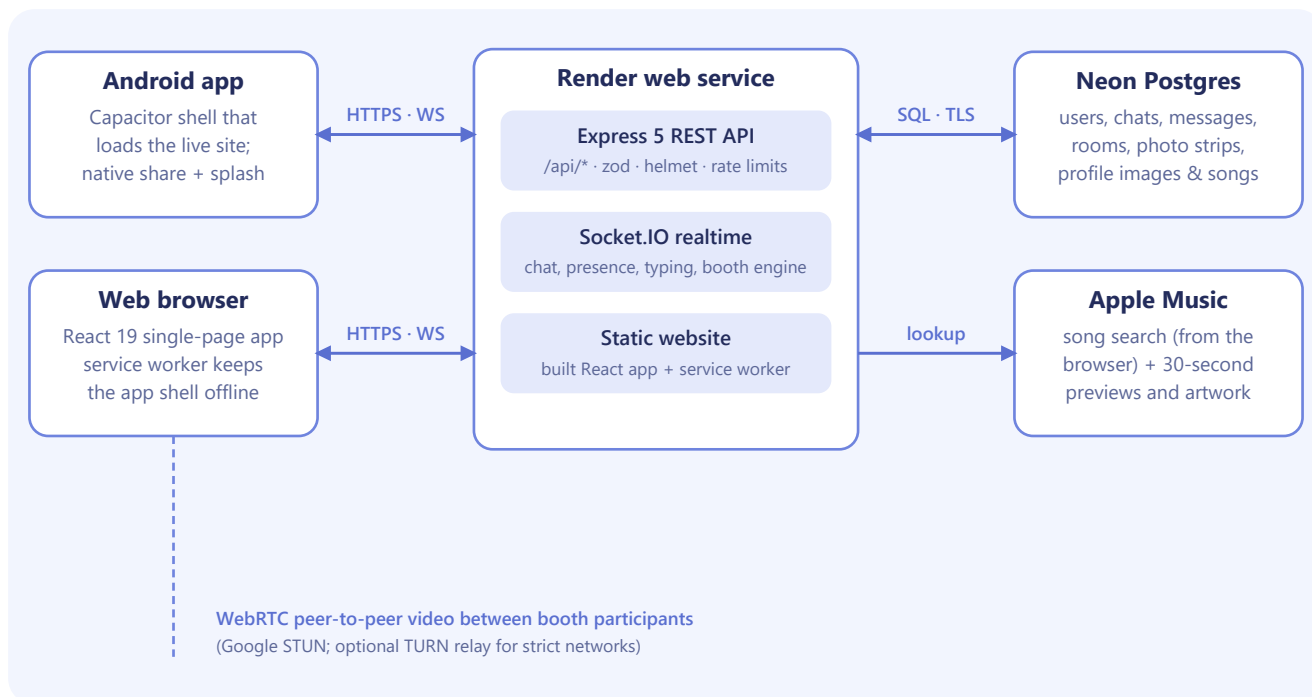
At a glance

Measure	Value
Source code	≈ 9,000 lines client (TypeScript/TSX/CSS) · ≈ 2,000 lines server (TypeScript/SQL)
REST endpoints	40 (auth 5, users 10, friends 4, conversations 10, rooms 5, photos 4, health/RTC 2)
Real-time events	10 client→server, 18 server→client (Socket.IO)
Database tables	9 (users, friendships, conversations, conversation_members, messages, booth_rooms, photos, profile_images, schema_migrations)
Migrations	3 SQL files, applied automatically at boot
Sticker library	216 stickers in 5 character packs
Booth	Up to 4 people, 1–6 shots, 1–10 s countdown, 3 layouts, 9 frames, 5 patterns, 7 filters
Main web bundle	481 KB (149 KB gzipped); photobooth code loads on demand (49 KB)
Hosting	Render web service (free plan, Singapore) + Neon Postgres (ap-southeast-1)
Android	Capacitor 8 app, min Android 7 (API 24), target API 36, ≈ 6.6 MB debug APK

SECTION 3

Architecture

One Render web service serves everything: the REST API, the real-time socket, and the built website. Data lives in Neon Postgres. Live video goes directly between participants' devices, so the server only coordinates.



- **Client:** React single-page app. Pages are routes (/chats, /friends, /booth, /memories, /profile, /u/:username, /booth/:code). Heavy photobooth code is split into separate chunks that load only when opened.
- **State:** Zustand stores — *auth* (signed-in user, wake-up state) and *chat* (conversations, messages, friends, presence, typing).
- **Server:** Express 5 for JSON APIs, Socket.IO on the same HTTP server for real time, and static hosting of the React build with an SPA fallback.
- **Real time:** each signed-in socket joins a private room *user:<id>*; the server pushes events to exactly the users involved (chat members, friends, booth participants).
- **In-memory state:** online presence and live booth sessions are kept in server memory (fast, but tied to a single instance — see section 23).

SECTION 4

Technology stack

Layer	Technology	Used for
Client UI	React 19, React Router 7, TypeScript	Screens, routing, type safety
Styling	Tailwind CSS v4 + hand-written CSS	Design tokens, light/dark themes, animations
Client state	Zustand 5	Auth and chat stores
Build	Vite 8, TypeScript 7	Dev server with API proxy, production bundles
Real time	Socket.IO 4 (client + server)	Messages, presence, typing, booth engine
Video	WebRTC (browser), Google STUN, optional TURN	Peer-to-peer booth video
Graphics	HTML Canvas 2D	Strip compositor, filters, QR card, image resizing
QR codes	qrcode	QR matrix with error correction level H
Server	Node 22, Express 5, helmet, compression, express-rate-limit	HTTP API, security headers
Validation	zod 4	Every request body and query

Layer	Technology	Used for
Auth	bcryptjs, jsonwebtoken, httpOnly cookies	Passwords and sessions
Database	PostgreSQL (Neon) via pg	All persistent data, including images
Music	Apple iTunes Search / Lookup API	Profile song search and previews
Android	Capacitor 8 (+ Share, Filesystem, Splash Screen)	Native app shell
Tooling	Python + Pillow (+ NumPy, SciPy)	Sticker slicing, icon generation
Hosting	Render (Blueprint render.yaml), Neon	Deploys from GitHub main

SECTION 5

Accounts & sign-in

Sign up and log in

What it does	Create an account with a username, email and password, and pick a buddy character. Log in with either the username or the email. A live check shows whether a username is free while you type, and a strength bar rates the password. “Remember me” keeps you signed in for 30 days; unticked, you're signed out when the browser closes.
How it's built	Username are 3–20 characters (letters, numbers, dots, underscores) and stored lower-case; passwords 8–128 characters, hashed with bcrypt (cost 11). On success the server sets an httpOnly <i>nivo_session</i> cookie holding a signed JWT (SameSite=Lax, Secure in production). Login compares against a dummy hash when the account doesn't exist so response time doesn't reveal which accounts exist. Register and login share a rate limit of 30 attempts per 15 minutes per IP.
How it performs	Username checks are debounced (350 ms) and cancelled when you keep typing, so the server sees one request per pause.

One-screen login page

What it does	A single page with Login / Sign Up tabs, the NivoTalk badge, “Snap. Smile. Share.”, and an animated photobooth illustration (the booth flashes and prints a strip, Dino snaps photos, the plant and curtain sway). On phones the form is a bottom sheet and the whole page always fits the screen with no scrolling.
How it's built	Built as an SVG illustration plus CSS keyframes. Sizes are tied to the screen height with CSS clamp() and media queries; on short phones extras (tagline, headline, divider) step aside, and the illustration shrinks to fill only the space left (hidden by a container query below 96 px). Google, Apple and “Forgot password?” are shown as designed but are placeholders (“coming soon”).
How it performs	Verified to fit without scrolling from 375×560 phones to 1536×860 desktops in both modes.

SECTION 6

Friends & presence

Friends list

What it does	Add people by username (or find them with search). See who added you (“Added you”), mark favourites, see who is a mutual friend, and see who's online or when they were last seen. Tap a friend to chat, start a photobooth, or open their full profile.
How it's built	Friendships are one-directional rows (user → friend), like adding someone by ID; “mutual” is computed with an EXISTS check. Search matches the start of usernames or display names (12 results). Presence is a per-user count of open sockets in server memory; when it goes 0→1 or 1→0 the server notifies everyone who added that person or shares a chat with them, and stores last-seen time on disconnect.
How it performs	Presence works across multiple tabs and devices per user (it only flips to offline when the last one closes).

SECTION 7

Chat

Direct and group chats

What it does	One-to-one chats and groups of up to 30. Groups can be renamed, members invited, and people can leave. Each chat can be muted. The chat list shows the last message, unread badges and who's typing.
How it's built	A direct chat is unique per pair through a <i>direct_key</i> (“smaller-id:larger-id”), so opening a chat twice reuses it. Group events (created, renamed, invited, left) are stored as system messages. The chat list is built with one query for the user's chats plus two batched queries for all members and last messages (up to 200 chats).
How it performs	Every action is pushed live to all members over Socket.IO, so lists and headers update instantly on every device.

Messages

What it does	Text (up to 4,000 characters, links become tappable), stickers, photo strips (with an optional caption) and photobooth invite cards. Messages show day dividers, delivery state, and “Seen” read receipts. Scrolling to the top loads older messages.
How it's built	Sending is optimistic: the message appears immediately with a client ID, is confirmed by the server, and can be retried if it fails. Read receipts use each member's <i>last_read_at</i> time. Typing indicators are sent over the socket and expire after 6 seconds. Pages hold 40 messages (max 100) using id-based pagination on an index of (conversation, id). Only http/https links are linkified, which blocks javascript: links.
How it performs	Scroll position is preserved when older messages load, and the view sticks to the bottom only when you're already near it.

SECTION 8

Stickers

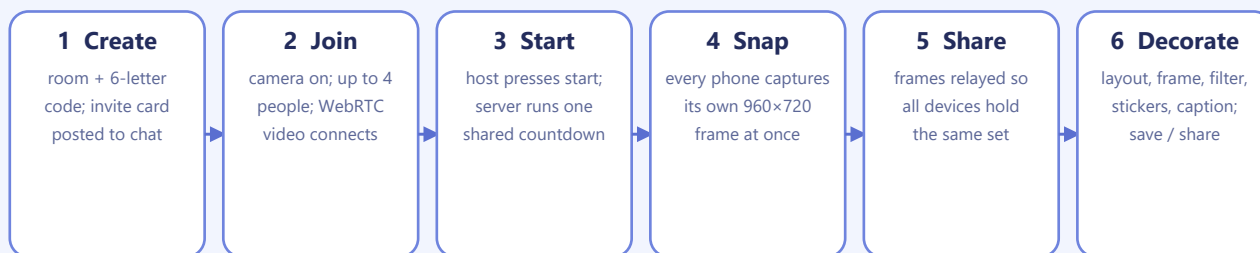
Sticker packs

What it does	216 expressive stickers across five characters — Berri, Chatty, Dino, Pepo and Pomi — usable in chat, as booth reactions and as strip decorations. Your 24 most recent stickers are remembered. Each character is also a “buddy” avatar used when someone has no profile photo.
How it's built	The packs come from illustrated sticker sheets in <i>assets/stickers</i> . A Python tool (Pillow, NumPy, SciPy) finds each character on the sheet, cuts it out with clean edges, saves it as WebP and writes a manifest (stickers.json) with ids, labels and sizes. The server only accepts sticker ids matching the known pack pattern.
How it performs	WebP stickers are small and cached by the browser; recent stickers are kept in local storage on the device.

SECTION 9

The live photobooth

The signature feature: up to four people, each on their own phone or computer, take one photo strip together on live video.



Server: Socket.IO booth engine (signaling relay, countdown timer, frame relay) · Video never passes through the server

Booth rooms

What it does	Create a room with a number of shots and a countdown, optionally posting an invite card straight into a chat. Friends join from the card, a link or the 6-letter code. A booth works solo too, and phones can flip between front and back cameras. While posing, anyone can send floating sticker reactions.
How it's built	Rooms are rows with a 6-character code from an alphabet without look-alike characters (no 0/O/1/I), a 24-hour expiry, and settings (1–6 shots, 1–10 s countdown). The live session lives in server memory: participants, settings and the current shoot. The host controls the shoot; if the host isn't there, the longest-present person does. Rooms hold at most 4 people.
How it performs	Rooms can be ended by the host; when the last person leaves, the in-memory session is cleaned up.

Live video and the synced countdown

What it does	Everyone sees everyone on live video. When the host presses start, the same countdown runs on every screen and every camera fires at the same moment.
How it's built	Video is WebRTC peer-to-peer (a mesh between up to 4 people); the server only relays the connection setup messages (offers, answers, ICE candidates) between members of the same room. Google STUN servers are used, and a TURN relay can be added through environment variables for strict networks. If a connection fails, the initiator restarts ICE automatically. The server drives the countdown itself: a 1.2 s "get ready", then for each shot a countdown event, the shot, and a 1.7 s pause for the flash and for frames to arrive.
How it performs	Cameras open at up to 1280×720. Video never passes through the server, so server load stays tiny no matter how long people stay in a booth.

Full-quality frames, shared

What it does	Instead of screenshots of a video call, each person's own camera captures a sharp photo, and everyone ends up with the same set of photos to build the strip.
How it's built	At each shot every device draws its camera frame to a 960×720 canvas (as the mirrored preview people saw) and encodes a JPEG (quality 0.88). It's sent over the socket and relayed to the others; the server rejects anything that isn't a JPEG data URL or is larger than ~1.8 MB. Frames are tagged with the session and shot number.
How it performs	A 4-person, 4-shot session moves about 16 frames of a few hundred KB each through the server — small and quick.

SECTION 10

Strip editor & Memories

Decorating the strip

What it does	After the shoot everyone gets an editor: choose a layout (Strip, Grid, or Double — two strips to cut and share), a frame colour (9 themes), a background pattern (plain, dots, gingham, hearts, sparkles), a filter (Original, Soft glow, Peach, Mono, Film, Cool, Pop), drag/resize/rotate stickers or tap “decorate” for a character preset, and add a caption and the date.
How it's built	A canvas compositor (composer.ts) lays out cells for the chosen layout and shot count, paints the frame, pattern, photos (with the filter applied as canvas operations), names, caption, date and stickers. The live preview draws the strip on a canvas with stickers as movable DOM overlays; export redraws everything at full resolution.
How it performs	Exports are JPEG at quality 0.92. The editor and booth code load only when a booth is opened.

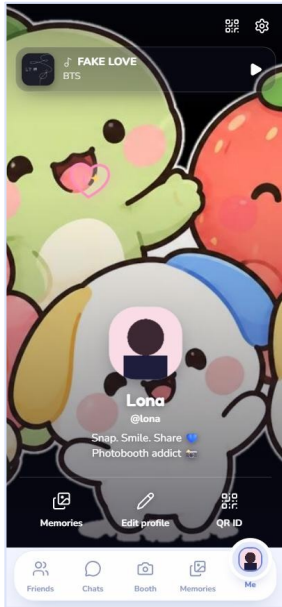
Download, share and Memories

What it does	Download the strip, send it into any chat, or save it to Memories — a private gallery of your strips, where you can open, download or delete them (deleting also removes it from chats you shared it in).
How it's built	Strips are uploaded as raw image bytes (up to 6 MB, up to 8000 px), identified by their actual bytes (JPEG, PNG or WebP only), and stored in Postgres. A strip is visible to its owner and to members of any chat where it was shared. In the Android app, downloads open the share sheet so people can save to Photos or send anywhere.
How it performs	Strips are about 300–600 KB each and are served with a one-year private cache header.

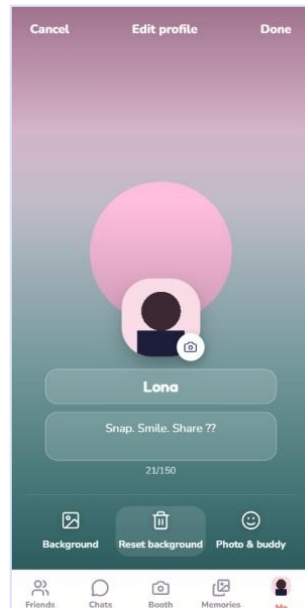
SECTION 11

Profiles, photos & bio

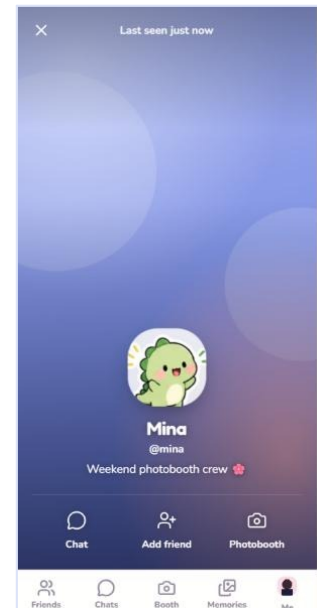
Profiles take their look from KakaoTalk: a full-screen background, a rounded profile photo, name, bio, and a row of actions.



Your profile



Edit mode



Someone else (/u/mina)

Profile screen

What it does	Your background photo fills the screen; your photo, name, @username and bio sit near the bottom above three actions: Memories, Edit profile and QR ID. A glowing heart floats over the background showing your friend count. Settings (light/dark mode, chat bubble colour, log out) open from the gear icon. On desktop the profile shows as a phone-shaped card over a blurred copy of the background.
How it's built	A shared ProfileStage component renders the background (or a soft gradient when there is none), shading for readable white text, and the animated entrance (background eases in, photo pops, text rises).
How it performs	Other people's profiles (/u:username) use the same screen with Chat, Add friend and Photobooth actions and update live when they change their photo or bio.

Edit profile

What it does	Edit in place: change the background, change or remove your profile photo, pick a buddy character for when there's no photo, and edit your name (up to 30 characters) and bio (up to 150, with a counter). Nothing is saved until you press Done.
How it's built	Pictures are prepared on the device before upload: decoded with the phone's rotation applied, profile photos centre-cropped to a square of at most 640 px, backgrounds scaled to at most 1440 px, transparent areas filled white, and saved as JPEG. The server checks the real file type, limits size (2 MB photo, 5 MB background), stores each image in <i>profile_images</i> , points the user at it, and deletes the image it replaced, all in one transaction. Uploads are limited to 30 per 10 minutes. Friends are notified live.
How it performs	Typical uploads are 30–150 KB. Each new image gets a new id, so its URL never changes and is cached for a year.

SECTION 12

Profile songs



Add a song



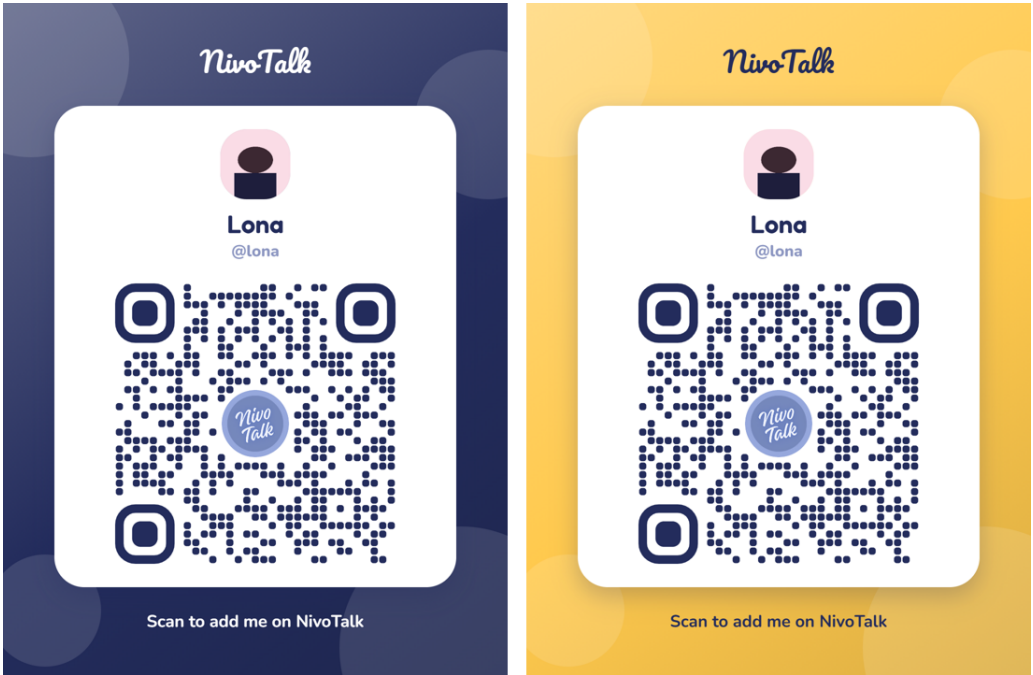
Song card on the profile

Pin a song, Instagram-style

What it does	In Edit profile, tap "Add a song", search any song or artist, tap a cover to hear a preview, and choose it. The song appears as a frosted card at the top of your profile with the album art, title and artist; anyone viewing your profile can play the 30-second preview, with dancing bars and a progress line while it plays.
How it's built	Search runs from the browser straight to Apple's public iTunes Search API, using the country from the phone's language setting so local music appears. When you save, the server looks the track up again with Apple's Lookup API and stores a snapshot (title, artist, artwork, preview URL) in <code>users.profile_song</code> ; only the track id is accepted from the client and only Apple https addresses are stored. A single shared audio player makes sure only one preview plays at a time and stops it when you leave the screen.
How it performs	Searching from each user's own device spreads Apple's rate limits across users instead of the server. Previews stream directly from Apple.

SECTION 13

QR ID



Exported QR cards in Midnight and Butter. The text colour adapts to the background.

Shareable QR card

What it does	Every user has a QR card: the NivoTalk wordmark, their photo, name and @username, and a QR code with the NivoTalk logo in the middle. Choose from 8 background colours or any custom colour; then Share (system share sheet), Download, or copy the profile link. Scanning it opens the person's profile with an Add friend button.
How it's built	The card is drawn on a 1080×1440 canvas. The QR code uses error correction level H (about 30% of the code can be covered), drawn as rounded dots with rounded corner markers; modules under the logo are left clear. The code itself stays dark-on-white so any scanner can read it, while the background and text adapt to the chosen colour. The chosen colour is remembered on the device.
How it performs	Verified with an independent QR decoder: all colour variants decode correctly at full size and when shrunk to 270×360 pixels.

SECTION 14

Design system, theme & motion

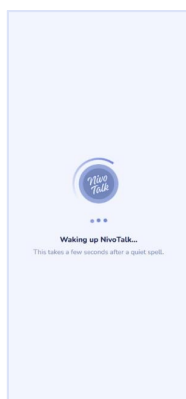
- **Brand colour:** periwinkle from the logo (#6F84DE primary, #5A6FD0 strong, #E4E9FB soft) on a pale #F2F5FE background, with navy #232C5C text. All colours are CSS variables, with a matching dark theme chosen in Settings.
- **Type:** Fredoka for headings, Nunito for text, Pacifico for the wordmark, Caveat for handwritten notes.
- **Logo everywhere:** one script (tools/make_app_icons.py) builds the Android icons, splash screens, favicon, Apple touch icon and in-app badge from the original logo file.
- **Tab bar:** on phones, a floating rounded bar; the active tab rises into a periwinkle bubble with a spring animation.
- **Motion:** entrance animations, floating hearts and sparkles, booth flash, sticker reactions, button shine. Everything respects the system “reduce motion” setting.

- **Mobile-first:** every screen was checked at phone sizes; safe areas (notches, gesture bars) are respected in the Android app and on iOS Safari.

SECTION 15

Server wake-up handling

Render's free plan puts the server to sleep after a while without traffic, and normally shows its own "waking up" page. NivoTalk hides that behind its own branded loader.

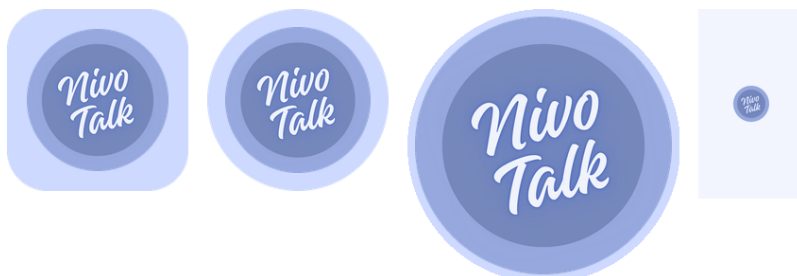


Branded loader while the server wakes

- **Loader:** the NivoTalk badge inside a spinning ring with bouncing dots, plus "Waking up NivoTalk..." when the wait is long. The same loader is written into index.html so it shows before any JavaScript loads.
- **Automatic retry:** while the API is unreachable or answers with Render's HTML page, the app keeps retrying the session check (gentle back-off) and then continues, still signed in. Tested by stopping and restarting the server: the app recovered about 5 seconds after the server returned.
- **Service worker:** keeps the last good copy of the app on the device. If the server is slow or answers with Render's page, returning visitors get the saved app (with its loader) instantly. It never caches API or socket traffic.
- **Android:** the native splash (the logo) stays up until the app's own loader is on screen, with a 60-second safety cap.

SECTION 16

Android app



Launcher icons (square, round, adaptive) and splash, generated from the real logo.

- **How it works:** a Capacitor shell (package com.nivotalk.app) that loads the live site, so every web deploy updates the app without reinstalling.
- **Native parts:** camera and microphone permissions for the booth, the system share sheet (save or send strips and QR cards), and the splash screen.

- **Build:** `npm run android:apk -w client` (Vite build → cap sync → Gradle assembleDebug) produces a ≈ 6.6 MB debug APK. A signed release build is needed for the Play Store.

SECTION 17

Data model

PostgreSQL schema, created by numbered SQL migrations that the server applies at start-up under an advisory lock.

Table	Holds	Key details
users	Accounts and profiles	username (unique, pattern-checked), email (unique), password_hash, display_name, status_message (bio), avatar (buddy), theme_color, last_seen_at, avatar_image_id, cover_image_id, profile_song (JSON)
friendships	Who added whom	(user_id, friend_id) primary key, favorite flag; indexed by friend_id
conversations	Chats	is_group, title, direct_key (unique per pair), created_by, updated_at for ordering
conversation_members	Chat membership	last_read_at (read receipts, unread counts), muted
messages	All messages	kind (text, sticker, photo, booth_invite, system), body, meta (JSON); indexed by (conversation, id) and by photo id
booth_rooms	Photobooth rooms	code (unique), host, linked chat, shots, countdown, status, expires_at
photos	Photo strips	owner, room code, mime, size, image bytes; indexed by (owner, created_at)
profile_images	Profile photos & backgrounds	owner, kind (avatar or cover), mime, image bytes
schema_migrations	Applied migrations	001_init, 002_profile_media, 003_profile_song

SECTION 18

API & real-time reference

REST endpoints (all under /api; everything except auth and health requires a session)

Area	Endpoints
Auth	POST /auth/register · POST /auth/login · POST /auth/logout · GET /auth/me · GET /auth/check-username
Users	PATCH /users/me · PUT, DELETE /users/me/avatar · PUT, DELETE /users/me/cover · PUT, DELETE /users/me/song · GET /users/images/:id · GET /users/search · GET /users/:username
Friends	GET /friends · POST /friends · PATCH /friends/:id (favourite) · DELETE /friends/:id
Chats	GET /conversations · POST /conversations/direct · POST /conversations/group · GET, PATCH /conversations/:id · POST /:id/members · POST /:id/leave · GET, POST /:id/messages · POST /:id/read
Booth	POST /rooms · GET /rooms/mine · GET /rooms/:code · POST /rooms/:code/invite · POST /rooms/:code/close
Photos	POST /photos (raw image) · GET /photos · GET /photos/:id · DELETE /photos/:id
System	GET /health (checks the database) · GET /rtc-config (STUN/TURN servers)

Socket.IO events

Direction	Events
Client → server	typing · booth:join · booth:leave · booth:signal · booth:settings · booth:start · booth:cancel · booth:frame · booth:react · (disconnect)
Server → client	message:new · conversation:new · conversation:updated · conversation:read · presence · typing · user:updated · booth:state · booth:peer-joined · booth:peer-left · booth:signal · booth:session-start · booth:countdown · booth:session-end · booth:session-cancel · booth:frame · booth:react · booth:closed

Sockets sign in with the same session cookie as the API; every chat and booth event checks membership before relaying.

SECTION 19

Security & privacy

- **Passwords & sessions:** bcrypt hashes; httpOnly, SameSite=Lax, Secure cookies; 30-day or browser-session lifetimes.
- **Input:** every request validated with zod; all SQL uses parameters (no string-built queries with user input).
- **Access control:** chat routes and socket events check membership; photo strips are only visible to the owner and members of chats they were shared in; profile images to signed-in users.
- **Uploads:** file type detected from the actual bytes (JPEG/PNG/WebP), size limits per kind, rate limits on uploads and song changes.
- **Headers:** helmet with a strict Content Security Policy (only self plus Google Fonts, Apple artwork/previews/search), camera and microphone limited to the site itself, no X-Powered-By.
- **Database connection:** TLS with certificate verification and channel binding for Neon.
- **Profile songs:** only Apple track ids are trusted from clients; stored URLs are re-fetched from Apple and limited to Apple https hosts.

SECTION 20

Performance

Area	How it's kept fast
Loading	Main bundle 481 KB (149 KB gzipped); photobooth (49 KB), booth home (11 KB) and memories (2.5 KB) load on demand; gzip compression on all responses.
Caching	Build files have content hashes and a one-year immutable cache; images get unique ids and long private caching; the service worker serves the app shell and assets from the device.
Video	Peer-to-peer, so the server carries no video; it only relays small signaling messages and one JPEG per person per shot.
Images	Resized on the device before upload (photos ≤ 640 px, backgrounds ≤ 1440 px, typically 30–150 KB).
Database	Connection pool of 10; indexes on the hot paths (messages by chat, friendships by friend, photos by owner); the chat list is three queries no matter how many chats.
Real time	Events go only to the users involved (per-user rooms), not broadcast.
Cold start	Free-plan sleep can make the first request take up to about a minute; users see the branded loader and the app resumes by itself.

SECTION 21

Build, run & deploy

Run locally

```
npm install
cp .env.example .env # DATABASE_URL (Neon) and JWT_SECRET
npm run dev          # API on :4000, website on :5173
```

Production build and Android

```
npm run build && npm start # serves API + site on :4000
npm run android:apk -w client # debug APK
python tools/make_app_icons.py # regenerate icons from the logo
npm run stickers # re-slice sticker sheets
```

- **Deploy:** Render Blueprint (render.yaml) — pushing to GitHub *main* triggers a build (npm ci, npm run build) and start; Render checks /api/health.
- **Configuration:** DATABASE_URL, JWT_SECRET (generated by Render), optional TURN_URL / TURN_USERNAME / TURN_CREDENTIAL, optional CLIENT_ORIGIN.
- **Migrations:** applied automatically on boot; each runs in a transaction and is recorded in schema_migrations.

SECTION 22

How NivoTalk was created

Commit	Milestone
8d2d1ad	Initial app: accounts, friends, chat with stickers, and the online photobooth with strip editor and Memories.
b6a4a83	Light theme by default and the Android app (Capacitor).
72f9131	Periwinkle redesign: new login/sign-up, Kakao-style profile with photo/background uploads, QR ID, /u/:username pages, new photobooth home and floating tab bar; working "Remember me".
0600084	Profile songs (Apple Music), branded wake-up loader with service worker and Android splash, and the real logo for every icon.

- **Approach:** designed screen by screen from mockups (login, profile, booth), built with an AI pair-programmer (Claude Code), and reviewed in a live browser preview at phone and desktop sizes after each change.
- **Testing:** type-checking and production builds on every change; the real server run against a local Postgres-compatible database (PGLite) to exercise endpoints end to end (uploads, limits, permissions, songs); UI flows driven in the browser; QR cards checked with an independent decoder; the APK inspected to confirm icons and plugins.
- **Assets:** stickers sliced from illustrated sheets; icons generated from the original logo file kept in assets/brand.

SECTION 23

Known limitations & roadmap

Area	Today	Suggested next step
Sign-in options	Google/Apple sign-in and password reset are placeholders.	Add OAuth providers and email-based reset.
Sessions	Tokens can't be revoked server-side; no "log out everywhere".	Session table or token version per user.
Abuse limits	Messages and photo strips have no rate limits or storage quotas; anyone can be added to a group.	Per-user rate limits and quotas; friends-only invites; block/report.
Scaling	Presence and live booths live in memory, so only one server instance; free-plan sleep.	Paid always-on instance; Redis adapter for multiple instances.
Storage	Images are stored in Postgres.	Move to object storage (S3/R2/Cloudinary) as usage grows.
QR links	Scans open the website even if the app is installed.	Android App Links for nivotalk.onrender.com.
Release	Debug APK; no automated tests or CI.	Signed release build for Play Store; integration tests on auth and permissions.